AZ-104 Lab 03 – Manage Azure Resources using ARM Templates & Azure Bicep

Lab Objective

This lab provides an in-depth understanding of Infrastructure as Code (IaC) in Azure using Azure Resource Manager (ARM) templates and Azure Bicep. It explains how to automate, standardize, and repeatedly deploy Azure resources using declarative templates.

Big Picture – Why This Lab Matters

As cloud environments grow, manual deployments through the Azure portal become error-prone and inconsistent. Infrastructure as Code enables teams to define infrastructure once and deploy it consistently across environments using automation. This lab forms the foundation for DevOps, CI/CD pipelines, and large-scale Azure deployments.

Azure Resource Manager (ARM)

Azure Resource Manager is the control plane for Azure. Every deployment, whether through the Azure portal, PowerShell, CLI, or Bicep, is processed by ARM. ARM validates requests, applies policies and RBAC, and ensures the desired state described in templates is achieved.

Declarative Deployment Model

ARM uses a declarative model. Instead of specifying step-by-step instructions, you declare the final desired state of the infrastructure. Azure determines the order of operations required to reach that state.

ARM Template Structure (Detailed)

$schema

Defines the schema URL used for template validation and IntelliSense support.

contentVersion

Used for versioning the template for tracking and change management.

parameters

Accepts input values at deployment time, allowing templates to be reused across environments.

variables

Stores calculated or reusable values within the template to reduce repetition.

resources

Defines the Azure resources to deploy, including type, API version, properties, and dependencies.

outputs

Returns values after deployment, useful for automation and chained deployments.

Lab Tasks – Step-by-Step Explanation

Task 1 – Create Resource & Export Template

A managed disk is created manually in the Azure portal. The resource configuration is then exported as an ARM template. This demonstrates that any Azure resource can be represented as code.

Task 2 – Modify and Redeploy ARM Template

The exported template is edited to change parameters such as disk name. Redeploying the template creates a new resource, showing how templates enable repeatable deployments.

Task 3 – Deploy Template using Azure PowerShell

Azure Cloud Shell with PowerShell is used to deploy the template. This demonstrates automation and script-based deployments suitable for administrators and Windows environments.

Task 4 – Deploy Template using Azure CLI

The same template is deployed using Azure CLI. This highlights cross-platform automation and its suitability for DevOps pipelines.

Task 5 – Deploy Resource using Azure Bicep

A Bicep file is authored and deployed. Bicep simplifies ARM authoring while still using ARM as the underlying deployment engine.

ARM Templates vs Azure Bicep (Detailed Comparison)

1. ARM templates use verbose JSON syntax which can be difficult to read and maintain.
2. Azure Bicep provides a concise, readable syntax with strong typing and IntelliSense.
3. Bicep files are compiled into ARM JSON before deployment.
4. ARM remains the core execution engine for all Azure deployments.
5. Bicep is recommended by Microsoft for new IaC development.

Interview-Ready Questions and Answers

Q: What is Infrastructure as Code (IaC)?

A: Infrastructure as Code is the practice of managing and provisioning infrastructure using declarative configuration files rather than manual processes.

Q: What is Azure Resource Manager?

A: Azure Resource Manager is the deployment and management service that enables consistent provisioning and governance of Azure resources.

Q: What is an ARM template?

A: An ARM template is a JSON file that declaratively defines Azure resources and their configuration.

Q: Why are parameters used in ARM templates?

A: Parameters allow templates to be reused across different environments and configurations.

Q: How does Azure Bicep differ from ARM templates?

A: Azure Bicep is a higher-level language that simplifies ARM template authoring and compiles into ARM JSON.

Q: How can ARM templates be deployed?

A: ARM templates can be deployed using the Azure Portal, PowerShell, CLI, and CI/CD pipelines.

Q: Why is IaC important for DevOps?

A: IaC enables repeatable, automated, and scalable infrastructure deployments, reducing errors and improving consistency.

Key Takeaways

Lab 03 establishes a strong foundation in Infrastructure as Code. ARM templates and Azure Bicep enable automated, consistent deployments. Bicep simplifies template authoring while ARM remains the underlying deployment engine. These concepts are essential for DevOps, CI/CD, and enterprise Azure environments.